



Building a Scalable WordPress Architecture using Infrastructure as Code

Heiko Graceffa - AWS Cloud Computing

Capstone Project 09.12.2025

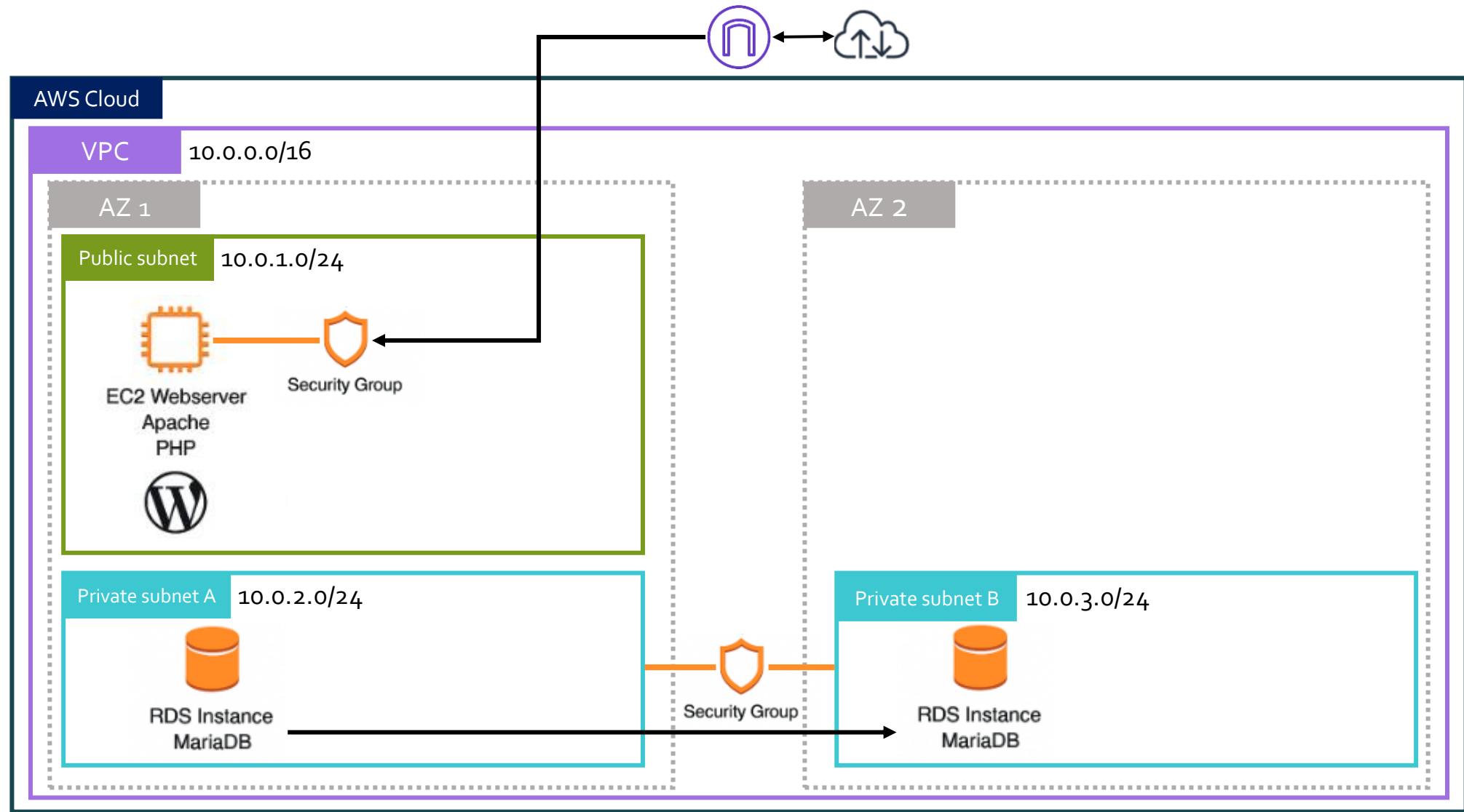
Agenda

- Problems & Requirements
- AWS Solution Architecture
- About the AWS-Services
- Infrastructure as Code (IaC): Terraform
- Deployment Workflow
- Result
- Q&A

Problems & Requirements

-  Manual steps are error-prone
-  Hard to scale or reproduce
-  No separation between web server and Database
-  Need a modern, automated WordPress solution
-  Should use scalable AWS services
-  Must be secure, repeatable, and cloud-native

AWS Solution Architecture



About the AWS-Services



Amazon **VPC** to provision a logically isolated network environment for all resources



Amazon **EC2** as a scalable compute service to deploy the WordPress webserver with Internet Gateway access



Amazon **RDS** to provide a managed and highly available MariaDB database that is not publicly accessible



Security Group to control traffic: allow HTTP/SSH only to the EC2 instance and allow MySQL access only from EC2

Infrastructure as Code (IaC): Terraform



Terraform as IaC for WordPress project that automates and manage the infrastructure more effectively.

```
9  variable "instance_type" {
10      description = "EC2 instance type"
11      type        = string
12  }
```

variables.tf

```
✓ resource "aws_instance" "ec2instance" {
    ami            = var.ami_id
    instance_type  = var.instance_type
    key_name       = var.key_name
    subnet_id      = aws_subnet.public_subnet.id
}
```

main.tf

```
1  aws_region      = "us-west-2"
2  ami_id          = "ami-04f9aa2b7c7091927"
3  instance_type   = "t3.small"
```

terraform.tfvars

Declaration of variables, in variables.tf files can be used in .tf files and can contain sensitive information that can be written in the terraform.tfvars file

- VPC + Subnets created with .tf files
- Security groups defined declaratively
- EC2 configured in a user_data.sh
- RDS created in Multi-AZ Mode
- Automatic insertion of RDS endpoint into WordPress config

Deployment Workflow

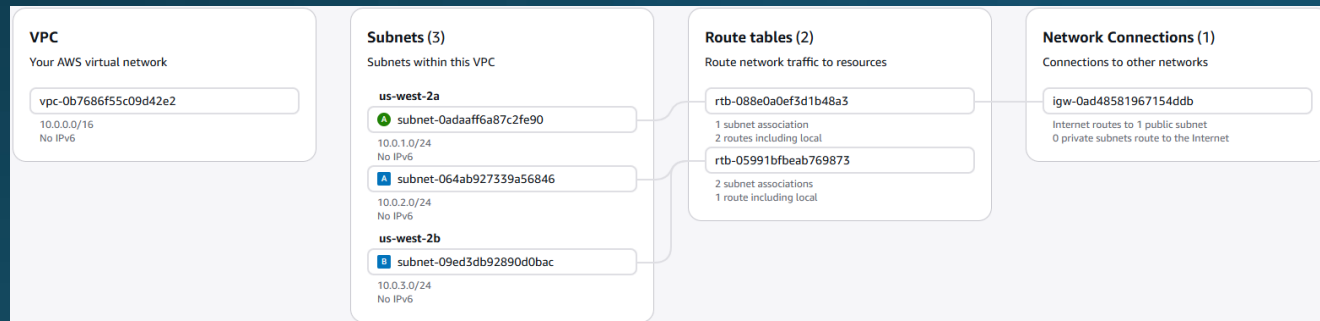
CI-like pipeline:

1. terraform init
2. terraform plan
3. terraform apply
4. EC2 user-data auto-installs WordPress
5. EC2 connects to RDS automatically

```
aws_db_instance.wordpress_db: Creation complete after 13m1s [id=db-ZZHZ35GEAID4462S4TB5F7LTS4]  
aws_instance.ec2instance: Creating...  
aws_instance.ec2instance: Still creating... [00m10s elapsed]  
aws_instance.ec2instance: Creation complete after 17s [id=i-01b24a147fa847403]  
  
Apply complete! Resources: 12 added, 0 changed, 0 destroyed.
```

Results

- EC2 webserver is running in the public subnet
- RDS MariaDB instance is available in private subnets (Multi-AZ)
- VPC networking configured correctly (IGW, subnets, route tables)
- Proper separation between web server and database
- All AWS resources provisioned automatically using Terraform



☐	Name ↗	Instance ID	Instance state ▼	Instance type
☐	wordpress-webserver	i-01b24a147fa847403	✓ Running 🔍 🔍	t3.small

☐	DB identifier ▲	Status ▼	Role ▼	Engine
☐	wordpress-db	✓ Available	Instance	MariaDB

Questions?